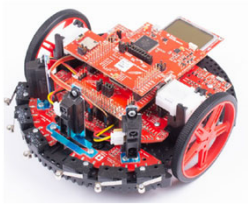
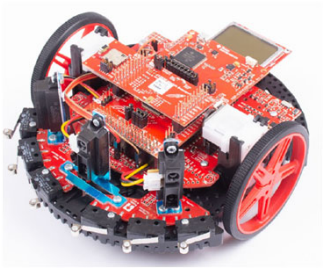
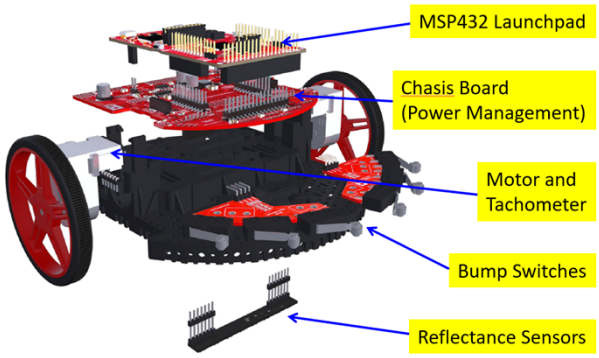
CE2107
Microprocessor System Design and Dev

Lab1 Briefing
Microprocessor Toolchain and C Programming



RSLK MAX Hardware





- TI RSLK MAX Robotic Kit
- MSP432 processor (ARM Cortex-M4F)
- Reflectance sensors
- Bump sensors
- IR sensors
- DC Motor
- Hall-Effect Tachometer

Source: Texas Instruments

- The hardware used for CE2107 Labs is the RSLK-MAX robotic kit from Texas Instruments.
- The main application processor is the MSP432 processor from TI, which is based on the ARM Cortex M4F processor core.
- The robot has a number of sensors which we will be using during the lab.
- These are
    - Reflectance Sensors that can be used to implement line following feature.
    - Bump sensors that detect collision.
    - IR Sensors that reports distance between the robot and an obstacle.
    - DC Motors and Hall-Effect Tachometer used to move the robot and track its speed respectively.

Power Management

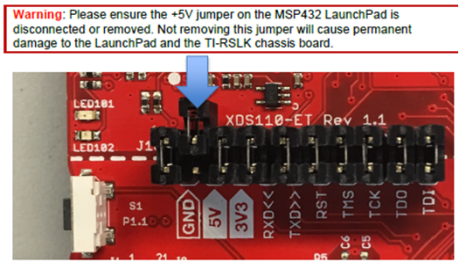


Figure 1



Figure 2

- Powered by 6xAA batteries.
- MSP432 Launchpad can also be powered via USB. But ensure that the 5V header is disconnected (Figure 1). This is to allow USB and Battery Power to be ON simultaneously.
- To power ON/OFF the hardware via batteries, toggle the Power button.
- Leave the Slider Switch in OFF position.

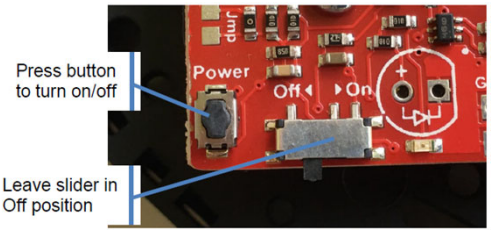


Figure 3

CE2107 – OHL 3

- The hardware is powered by 6 AA batteries.
- When you use the hardware, please check that 5V header on the MSP432 launchpad is disconnected, see Fig1 in the slide for illustration.
    - This is important and needed to isolate the USB 5V supply from the Battery Supply, so that both can be used simultaneously.
- Press the power button to toggle and hardware ON/OFF. See Fig3 for illustration.
    - Keep the slider switch in OFF position all the time. There is no need to use this switch.

Lab Topics

- Lab1 – Microprocessor Toolchain and Embedded System C Programming.
- Lab2 – General Purpose Input/Output (GPIO) and Finite-State Machine (FSM) Design.
  - Reflectance Sensor Interface (GPIO).
  - FSM with Reflectance Sensor Data input.
- Lab3 – Exception Handling and Timer Compare Operation.
  - Bump Switch Interface via Systick (Polling), and GPIO edge-triggered interrupt.
  - DC motor speed control via PWM signal generated from Timer configured in Compare mode operation.
- Lab4 – Analog-to-Digital Convertor (ADC) and Timer Capture Operation.
  - IR Sensor interface via ADC.
  - Tachometer interface via Timer configured in Capture Mode to measure period of incoming square pulses from Tachometer.
- Lab5 – Lab Assessment
  - Integration of work done in Lab2 to Lab4.

- This slide contains the main topics that we will explore in each lab. You can pause the video to go through the text.
- Note that the labs are designed to be cumulative in nature, work done in each lab will be needed for subsequent labs so make sure you complete the previous lab before embarking on the one.
- Lab5 is used for lab assessment purpose. Work done in Lab1 to Lab4 will be needed for assessment task that you need to do in Lab5.
- A separate set of briefing slides will be created to go through the details on the assessment during Lab5.

Useful Reference

- On top of the recommended text, below are some documents which will be very useful as a knowledge base.
- Uploaded to NTULearn under Reference Doc Folder
- MSP432 Technical Reference Manual (slau356h.pdf)
- MSP432 Datasheets (msp432p401r.pdf)
- Coretx M4 Generic UG (Cortex-M4-GenericUG-DUI0553.pdf)
- All reference documents can also be found in the 'Doc' directory of the zip file.
- The original RSLK Max workshop is also available online at https://university.ti.com/en/faculty/ti-robotics-system-learning-kit/ti-rslk-max-edition-curriculum

- Some useful documents that you will need to refer to for your labs.
- Two main doc are the slau356 MSP432 TRM and the MSP432 Datasheets.
- You will need to refer to these doc for details of the control, status and data registers that you need to use for various labs.
- And the configuration information for various peripherals such as the GPIO, Clock, Timers, NVIC etc.

Preparation for Lab

- Download lab related materials from NTULearn Lab folder.
  - Lab Manuals
  - Lab Handouts to be filled in after each lab. Graded assignment.
  - Zip file containing all the codes required for the five labs. Include all reference documents mentioned in previous slide.
- Go through relevant chapters of the lecture materials.
- Note that consistent work is required and the labs are cumulative, i.e. current lab required work done in previous labs.

- Download the materials listed in this slide before coming to the lab.
- The handout mentioned here has a list of questions related to the lab, you need to fill in the answers and hand in before the next lab.
- Handout formed part of the lab assessment, in addition to the Lab5 assessment task.
- Note again that the lab task are cumulative so consistent work is required.

Lab1 Overview

- Lab1 consist of a series of instructions to let students familiarise with the following topic.
  - Code Composer Studio (CCS) IDE Introduction
  - C Language Programming for Embedded System
  - ARM Assembly Language analysis
  - Linker tool
  - RSLK Self Test demo
  - **Assessment Handouts. This is the easiest component to score, so do not miss out on this.**
- Very little coding is required (if any) for Lab1. Majority of the work is analysis and learning important skill sets required to carry out your tasks in Lab2 to Lab5.

- The slide here list the main topics that you will explore in Lab1.
- Lab1 is an introduction to the hardware and software that you will be using, and some of the key C programming techniques that you need to use for subsequent labs.

CCS IDE Introduction

- Detail instructions in the lab manual to bring you through various process.
  - **CCS environment setup (section 6.4)**
    - Connecting CCS to the RSLK MAX hardware.
    - Installing the CCS project sources downloaded from NTULearn.
    - Setting up a workspace in CCS.
    - Importing existing CCS projects in the zip file to the workspace created.
  - **CCS IDE features (Section 7.3, 7.4)**
    - Editor and Debugger mode.
    - Functions and tools available in these modes.
    - Observing status of core and peripherals registers.

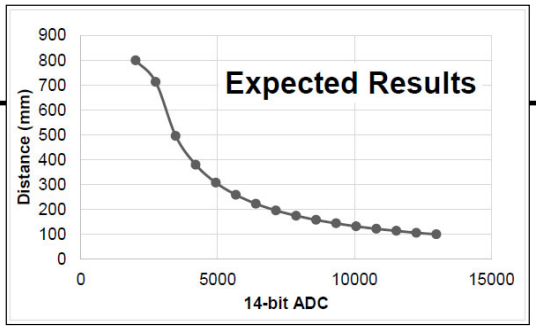
- First section is on CCS IDE introduction.
- You will be introduced various features of CCS via a guided tour using sample projects. Pay attention especially to the features used for debugging as you will need these later when debugging your code. The main value proposition of a commercial IDE such as CCS, compared to simpler IDE such as Arduino, is in the features available to allow insights and control to the processor operation, such as code stepping, breakpoints, register/variables watch windows, memory browser, disassembly window etc.

C Language Programming

- **Section 7.4.**
- A few important techniques commonly used in embedded system development is introduced in this sub section.
  - **Bit Manipulation**
    - How to change the value of a few specific bits in a register. E.g. how to clear bit0, bit3 and bit5 of the P1SEL0 register. Without touching the other bits in the register.
  - **Bit shifting and extraction**
    - How to pack specific bits in a variable to align with the right edges (LSBs). E.g. shifting bit 5 and bit6 of a variable to occupy bit0 and bit1 position and clear the rest of the original bits in the variable.
  - **Volatile qualifier keyword**
    - The effects of assigning the volatile qualifier to a variable.
- Some **C directives** to allow conditional compilation is introduced: #if, #else and #endif.

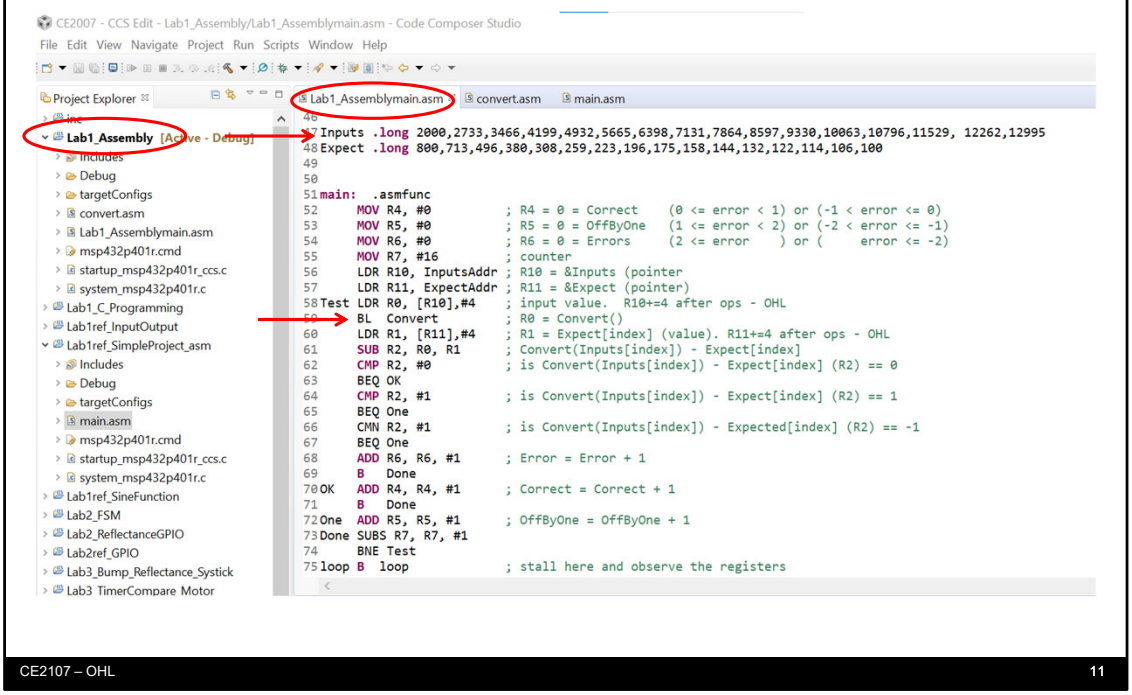
- The labs project codes uses C so it will be good for you to know some common techniques employed when doing embedded system C programming.
- This section on C programming introduce you some of these techniques.
- You can pause the video here to go through the list and their corresponding details.
- The lab manual has detail guidance to bring you through these topics so I will not elaborate further here.

Assembler



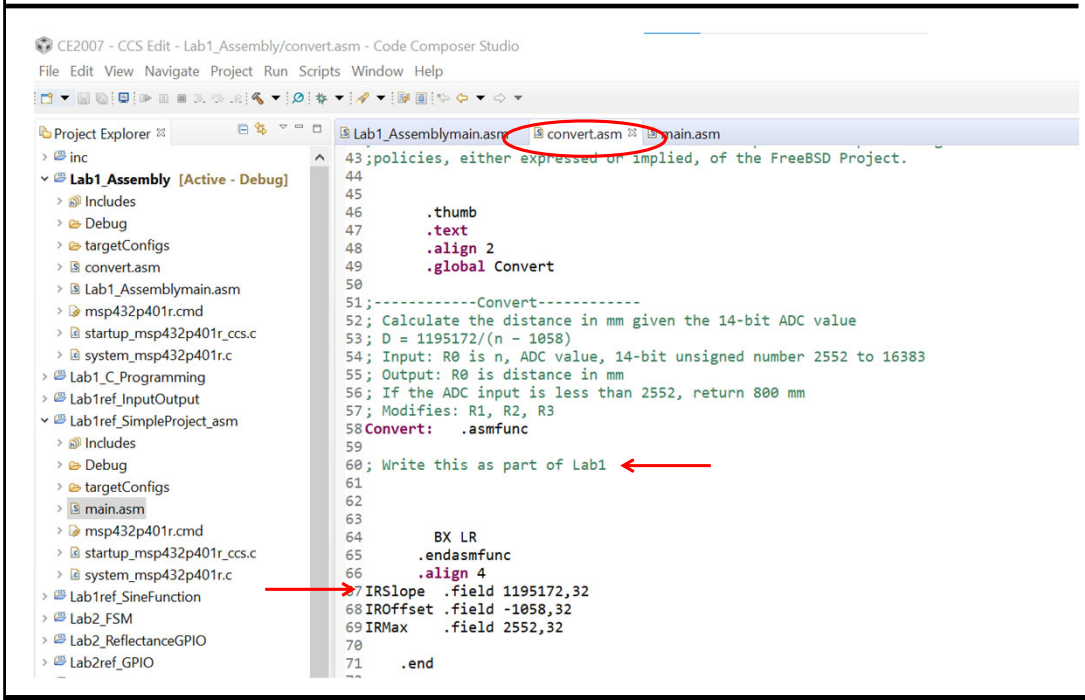
- **In section 7.5**, you will code a short assembly routine to convert the digital output from a ADC to derive the corresponding distance it represent.
- The 'convert' routine you code will take in an input and covert to distance value using the following expression.
- D = 1195172/(n – 1058)
- D = Distance and n = input
- The convert routine is called from a main routine which will also check if the converted value is correct based on some test vectors.
- Note that do not use the graph above to check if your converted value is correct. The required test vectors are in the main routine.
- More details in subsequent slides.

Assembler (Section 7.5)



- The areas of interest in this slide is indicated by the red circle and red arrows.
- In the Lab1_Assemblymain.asm file, the man routine calls the Convert routine via BL instruction.
- The Convert routines takes in a series of input from the array ‘Input’ indicated by the red arrow at the top of the file.
- The .long assembler directive reserved 32-bit data.
- The returned value of the Convert routine is then compared with values in the ‘Expect’ array to see if the results are correct.
- The number of correct and wrong results are then tracked in two registers.
- You will be able to know if the Convert routine you code has implemented the expression correctly or not by checking the two registers to see the correct and wrong counts. Which registers are used is not revealed here, you will need to check the main routine code to find out.

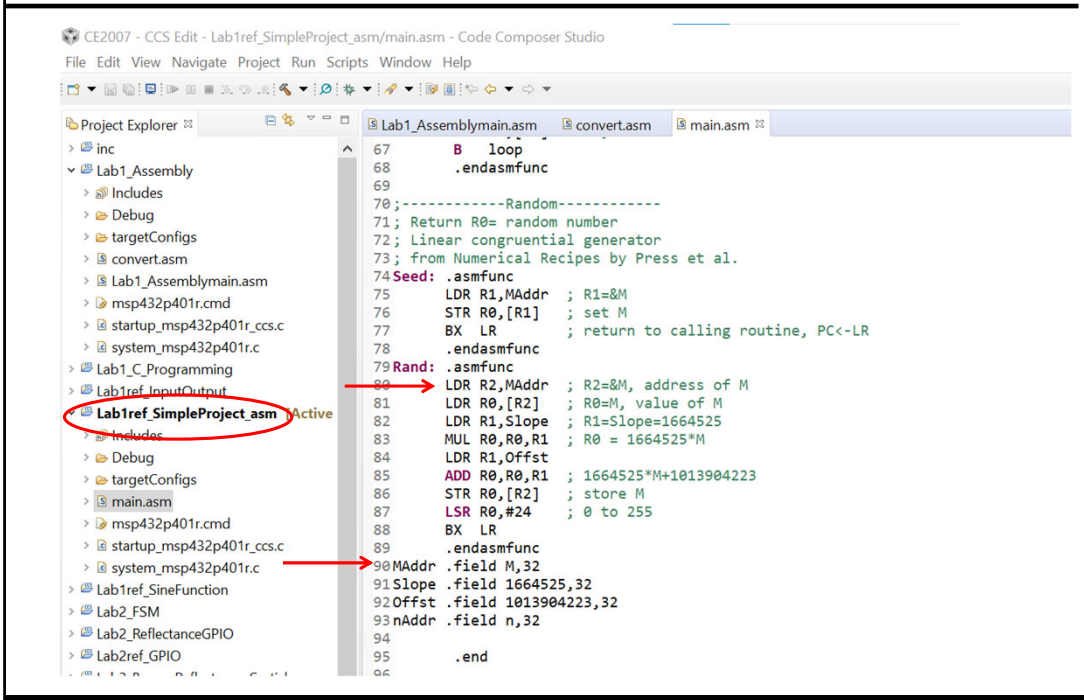
Assembler (Section 7.5)



CE2107 – OHL 12

- This slide shows you the convert.asm file which you need to modify to code the convert routine.
- Note that the Assembler in CCS does not support the pseudo instruction LDR Rx, =LONG_CONSTANT used to initialise ARM register with 32-bit immediate constant. Instead, the directive .field is used to store the 32-bit constant required and the attached label e.g. IRSlope is used to transfer the 32-bit data into the data register. Check the next slide for sample code of how this is done.

Assembler (Section 7.5)



- This main.asm file is taken from the Lab1ref_SimpleProject_asm project.
- The two lines indicated by the red arrow show how a 32-bit constant M can be assigned to R2.

Assembler and Linker

- **Assembler (Section 7.5)**
  - Use CCS Disassembly Window to view the corresponding assembly functions used to implement each C statement.
  - Understand how functions exchange data according to the AAPCS (ARM Architecture Procedure Call Standard).
- **Linker (Section 7.6)**
  - Look into the map file to understand how C compiler works with the linker to allocate code/data to different software sections.
  - Learn how to use map file to find out the memory utilisation profile of a program.
  - Hands-on validation of the Compiler-Linker memory allocation concept learn in lecture.

- Each C statement is implemented with multiple assembly instructions.
- In section 7.5, you will also be tasked to explore how C compiler translate each C statements to multiple assembly instructions.
- In section 7.6, you will look into the function of a linker and the various useful information that can be extracted from the linker command file and map file.
- Do read up the corresponding lecture notes before coming to the lab.

RSLK-MAX Self Test

- The last **section (7.7)** allows you load a self test code without having the source code project.
- This is know as a project-less code download.
- The RSLK-MAX-SelfTest.out file used in this section is a self test application that communicates with the PC via USB Virtual COM Port.
- User can issue command via a terminal window to check various hardware on the RSLK-MAX robotic kit.
- This is also the basic application you need to develop for the Lab5 assessment.
- You can also use this sample .out application to verify if the RSLK-MAX hardware you have is functioning properly or not during your code development.